

UNIVERSIDAD DEL PACÍFICO
FACULTAD DE INGENIERÍA



AEROMETRIX GLOBAL FORECAST

Análisis comparativo de datos climáticos históricos y en tiempo real para múltiples países

Documento técnico final

Curso	Programación Avanzada para la Ciencia de Datos
Sección	A-PRE-PRE2026_IPER-170107
Proyecto	AeroMetrix Global Forecast
Repositorio	https://github.com/manuelsialer/AeroMetrix-Global-Forecast
GitHub	
Ejecución local	streamlit run src/app.py
Fecha	
24 de junio de	
2026	

INTEGRANTES:

Cristhian Salvador Rivas: Chamorro cs.rivasch@alum.up.edu.pe

Luis Fernando Apolaya Alva: lf.apolayaa@alum.up.edu.pe

Manuel Antonio Sialer: Flores ma.sialerf@alum.up.edu.pe

Salvatore Filiberto Muñoz: Chavez sf.munozc@alum.up.edu.pe

DOCENTE: Oswaldo José Velasquez Castañon

Resumen ejecutivo

AeroMetrix Global Forecast es una aplicación de ciencia de datos desarrollada en Streamlit para analizar información meteorológica de múltiples ciudades. La versión final amplía el prototipo parcial basado en CSV hacia un flujo completo de extracción externa, persistencia SQL, procesamiento analítico, visualización interactiva y control de calidad de código. El sistema consulta datos actuales de OpenWeatherMap, descarga series históricas desde Open-Meteo Archive, almacena la información en Supabase/PostgreSQL y presenta indicadores, mapas, tendencias temporales, comparaciones por radar, registro de eventos y una proyección simple de temperatura mediante Random Forest (Open-Meteo, n.d.; OpenWeatherMap, n.d.; Scikit-learn developers, n.d.; Streamlit, n.d.; Supabase, n.d.).

El enfoque técnico responde a la rúbrica final: datos externos documentados, esquema relacional, consultas integradas, transformaciones relevantes, dashboard navegable, controles de usuario, manejo de errores, decoradores, estructura modular y documentación crítica del proyecto.

1. Objetivo y alcance de la entrega final

Objetivo: Construir una app de ciencia de datos que transforme datos atmosféricos externos en información accionable para comparar ciudades, monitorear condiciones recientes y explorar tendencias climáticas históricas.

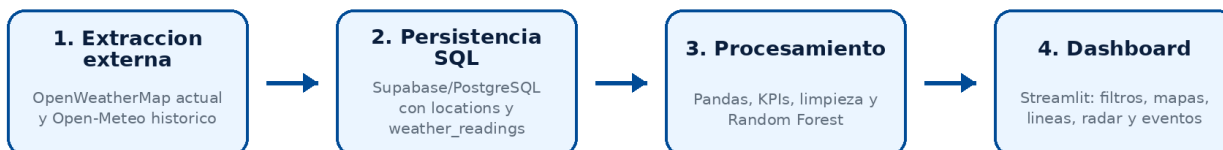
Requisito final	Implementación en AeroMetrix	Evidencia técnica
API / scraping funcional	APIs REST: OpenWeatherMap para clima actual y Open-Meteo Archive para histórico horario.	src/api.py, scripts/seed_5years.py
Persistencia SQL	Supabase/PostgreSQL con dos tablas relacionadas y restricciones UNIQUE.	init_db.sql, src/db.py
Dashboard completo	Streamlit con sidebar, tabs, KPIs, gráficos Plotly, mapa, radar y registro de eventos.	src/app.py, src/viz.py
Procesamiento/análisis	Limpieza de respuesta SQL, KPIs, deltas y modelo Random Forest para temperatura.	src/processing.py
Calidad de código	try/except, decorador de tiempo, caché, módulos separados y pruebas básicas.	src/db.py, src/app.py, tests/

2. Flujo de datos: API/scraping -> SQL -> dashboard

El flujo parte de consultas HTTP a servicios externos, convierte las respuestas JSON a una estructura normalizada, guarda las observaciones en una base relacional y luego consulta solo el

Subconjunto necesario para visualizaciones interactivas. Esta separación evita que el frontend dependa de descargas masivas en cada interacción.

Flujo técnico: API / archivo historico -> SQL -> procesamiento -> dashboard



Controles de robustez: validacion Pydantic, st.cache_data, try/except, UNIQUE constraints, upsert por lotes y decorador de tiempos.

Figura 1. Flujo técnico final de AeroMetrix.

3. Fuentes externas de datos

Fuente	Uso	Parámetros / autenticación	Control de robustez
OpenWeatherMap Current Weather API	Clima actual por ciudad: temperatura, humedad, viento, descripción, coordenadas y país.	Endpoint /data/2.5/weather; q, appid, units=metric, lang=es. Variable de entorno operativa: WEATHER_API_KEY.	response.raise_for_status(), validación Pydantic, cache TTL 600 s y try/except de validación.
Open-Meteo Archive API	Histórico horario por coordenadas para completar series de análisis.	Endpoint archive-api.open-meteo.com/v1/archive; latitude, longitude, start_date, end_date, hourly y timezone=UTC. No requiere llave.	Conversión de códigos WMO a descripción climática, omisión de temperaturas nulas y carga por lote.
Cron worker interno	Recolección pasiva para ciudades ya registradas y evaluación de alertas de viento.	schedule.every(15).minutes; pausa time.sleep(2) entre ciudades.	Evita saturar cuotas, maneja excepciones por ciudad y simula webhook de alerta.

4. Persistencia SQL y diseño de base de datos

La persistencia usa Supabase como servicio PostgreSQL. El modelo mantiene una tabla maestra de ubicaciones y una tabla transaccional de lecturas climáticas. La relación location_id permite integrar ciudad, país y coordenadas con cada observación meteorológica sin repetir metadatos geográficos.

Diseno de base de datos relacional

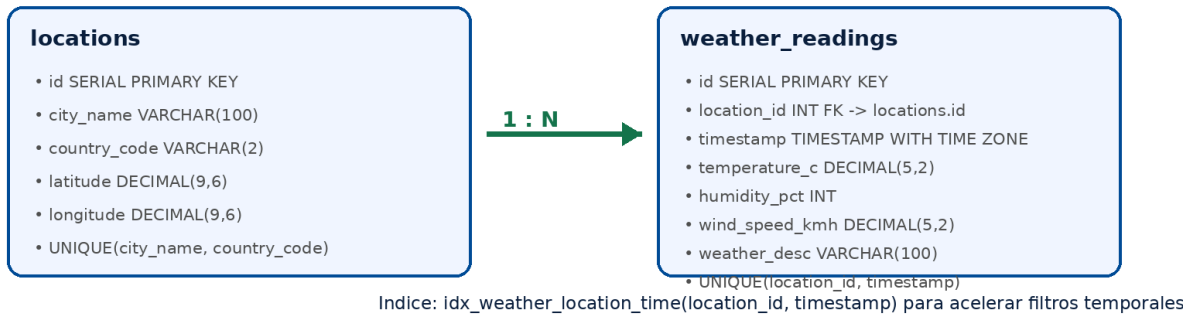


Figura 2. Esquema relacional propuesto e implementado en init_db.sql.

Consultas integradas: `get_available_cities()` carga ciudades para los filtros; `get_historical_data()` aplica filtros por `location_id`, `start_date`, `end_date`, ordenamiento descendente y `limit` para proteger la app de consultas excesivas. Para ingesta masiva, `batch_save_weather_readings()` usa bloques de 1000 registros y `upsert` con `ignore_duplicates`.

5. Procesamiento, métricas y análisis

Componente	Descripción	Resultado en el dashboard
Normalización de datos	<code>process_raw_db_data()</code> aplanar la respuesta anidada de Supabase, extrae campos de <code>locations</code> y convierte <code>timestamp</code> a <code>datetime</code> .	DataFrame limpio para tablas, KPIs y gráficos.
KPIs climáticos	<code>calculate_kpis()</code> calcula último registro y deltas contra promedios históricos de temperatura, humedad y viento.	Tarjetas superiores con valor reciente, variación y condición climática.
Modelo simple	<code>train_and_predict_temp()</code> usa <code>RandomForestRegressor</code> con hora cíclica, humedad y viento; valida con <code>train/test split</code> .	Proyección a 24 horas y métricas RMSE/MAE cuando hay datos suficientes.
Comparación geográfica	<code>create_radar_chart()</code> normaliza temperatura, humedad y viento para comparar dos ciudades.	Radar comparativo en pestaña dedicada.
Eventos	El dashboard clasifica lecturas recientes como viento alto, temperatura extrema o estado óptimo.	Tabla de eventos con estación, magnitud y estado.

6. Dashboard Streamlit: navegación, visualizaciones y controles

La interfaz opera con un sidebar de configuración y un área principal con pestañas. Los controles no solo cambian la visualización: también afectan las consultas enviadas a Supabase, reduciendo datos innecesarios antes de graficar.

Zona / pestaña	Controles o visualizaciones	Finalidad analítica
Sidebar - filtros	multiselect de ciudades, date_input de rango y multiselect de métricas.	Comparar ciudades y limitar consulta por rango temporal.
Motor de datos	selectbox de país/ciudad, text_input para ciudad nueva, botón Escanear Ahora y botón Descargar Historial.	Ingestar clima actual o histórico desde APIs externas.
Monitoreo Global	Gráfico de líneas y mapa Scatter Mapbox por métrica.	Ver evolución temporal y distribución espacial.
Machine Learning	Entrenamiento y gráfico de predicción de temperatura.	Explorar tendencia futura aproximada.
Radar Comparativo	selectbox Ciudad 1 y Ciudad 2, radar normalizado.	Comparar perfiles climáticos.
Registro de Eventos	DataFrame de últimas lecturas con tipo de evento y estado.	Auditar condiciones recientes relevantes.

7. Calidad de código, estructura y reproducibilidad

El repositorio mantiene una estructura modular coherente con la guía del curso: src/app.py concentra la interfaz; src/api.py gestiona extracción externa; src/db.py centraliza conexión, consultas y escritura SQL; src/processing.py contiene KPIs y modelo; src/viz.py encapsula gráficos; scripts/ automatiza ingesta y tests/ valida lógica crítica.

Criterio	Implementación
Manejo de errores	try/except en llamadas API, carga CSS, escritura SQL, worker y descarga histórica.
Decoradores	timer_decorator registra el tiempo de ejecución de funciones SQL; st.cache_data reduce consultas repetidas.
Modularidad	Separación de responsabilidades por módulos y scripts.
Pruebas	tests/test_processing.py valida KPIs con DataFrame vacío y datos controlados.
Ejecución	README incluye entorno virtual, pip install y streamlit run src/app.py.

8. Referencias

Open-Meteo. (n.d.). *Open-Meteo documentation*. <https://open-meteo.com/>

OpenWeatherMap. (n.d.). *OpenWeatherMap API documentation*. <https://openweathermap.org/api>

Scikit-learn developers. (n.d.). *Scikit-learn documentation*.
<https://scikit-learn.org/stable/documentation.html>

Sialer Flores, M. A., Rivas Chamorro, C. S., Apolaya Alva, L. F., & Muñoz Chavez, S. F. (2026).
AeroMetrix Global Forecast [Código fuente]. GitHub.
<https://github.com/manuelsialer/AeroMetrix-Global-Forecast>

Streamlit. (n.d.). *Streamlit documentation*. <https://docs.streamlit.io/>

Supabase. (n.d.). *Supabase documentation*. <https://supabase.com/docs>

The pandas development team. (n.d.). *pandas documentation*. <https://pandas.pydata.org/docs/>

11. Declaración de uso de IA

Se utilizó asistencia de IA para organizar, sintetizar y revisar la redacción del documento técnico final. El equipo mantiene la responsabilidad sobre el código, las credenciales, la ejecución del prototipo, la verificación de fuentes externas y la entrega final en Blackboard.